

INF.04 — Przygotowanie do egzaminu

SORTOWANIE

Algorytmy sortowania krok po kroku

Bubble Sort • Selection Sort • Insertion Sort • Merge Sort • Quick Sort

1. Wstęp.

1.1. Dlaczego sortowanie jest ważne?

- Posortowane dane łatwiej przeszukiwać — zamiast przeglądać wszystko po kolei, możemy używać szybszych metod (np. wyszukiwanie binarne).
- Sortowanie jest podstawą wielu innych algorytmów.
- W codziennym życiu sortowanie jest wszędzie: listy kontaktów w telefonie, wyniki wyszukiwania, zakupy sortowane po cenie.

1.2. Jak mierzymy efektywność algorytmów sortowania?

Każdy algorytm sortowania można ocenić pod kątem dwóch rzeczy:

Kryterium	Co oznacza	Przykład
Złożoność czasowa	Ile operacji wykonuje algorytm	$O(n^2)$ oznacza, że dla 100 elementów może być 10 000 operacji
Złożoność pamięciowa	Ile dodatkowej pamięci potrzebuje	$O(1)$ oznacza, że nie potrzebuje prawie żadnej dodatkowej pamięci
Stabilność	Czy zachowuje kolejność równych elementów	Ważne np. przy sortowaniu listy osób po nazwisku

i Notacja dużego O — wyjaśnienie prostymi słowami

$O(1)$ — czas stały: nie zależy od liczby elementów (błyskawiczne)

$O(n)$ — czas liniowy: podwójnie więcej danych = dwukrotnie więcej czasu

$O(n \log n)$ — czas logarytmiczny: dobre algorytmy sortowania

$O(n^2)$ — czas kwadratowy: podwójnie więcej danych = 4 razy więcej czasu

2. Sortowanie bąbelkowe (Bubble Sort)

2.1. Idea algorytmu

Sortowanie bąbelkowe to najprostszy algorytm sortowania. Jego nazwa pochodzi od zachowania algorytmu — duże wartości wypływają na koniec tablicy jak bąbelki powietrza w wodzie.

Zasada jest prosta:

Przechodzimy przez tablicę wielokrotnie. Za każdym razem porównujemy dwa sąsiednie elementy. Jeśli są w złej kolejności — zamieniamy je miejscami. Powtarzamy to tak długo, aż żadna zamiana nie jest już potrzebna.

Analogia z życia

Wyobraź sobie, że stoisz w kolejce 5 osób ustawionych według wzrostu, ale losowo. Porównujesz osobę 1 z osobą 2 — jeśli pierwsza jest wyższa, zamienią ją miejsca. Potem porównujesz osobę 2 z osobą 3, potem 3 z 4, potem 4 z 5. Po pierwszym przejściu NAJWYŻSZA osoba stoi na końcu — to był pierwszy 'bąbel'. Powtarzasz to od początku, ale już bez ostatniej osoby (ona już jest na miejscu). Kontynuujesz, aż wszyscy stoją w odpowiedniej kolejności.

2.2. Przykład krok po kroku

Mamy tablicę: [5, 3, 8, 1, 4]. Chcemy ją posortować rosnąco.

PRZEJŚCIE 1 (i = 0):

Krok 0	5	3	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 0 i 1: $5 > 3 \rightarrow$ zamieniamy miejscami.

Po zamianie	3	5	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 1 i 2: $5 < 8 \rightarrow$ bez zamiany.

Bez zmian	3	5	8	1	4
indeks	0	1	2	3	4

Porównujemy indeks 2 i 3: $8 > 1 \rightarrow$ zamieniamy miejscami.

Po zamianie	3	5	1	8	4
indeks	0	1	2	3	4

Porównujemy indeks 3 i 4: $8 > 4 \rightarrow$ zamieniamy miejscami.

Po zamianie (koniec przejścia 1)	3	5	1	4	8
indeks	0	1	2	3	4

✓ Po przejściu 1

Tablica: [3, 5, 1, 4, 8]

Element 8 jest już na swoim miejscu — największy trafił na koniec!

W następnym przejściu nie musimy sprawdzać ostatniego elementu.

PRZEJŚCIE 2 ($i = 1$) — porównujemy tylko pierwszych 4 elementów:

Przed przejściem 2	3	5	1	4	8
indeks	0	1	2	3	4

$3 < 5 \rightarrow$ bez zamiany. $5 > 1 \rightarrow$ zamiana. $5 > 4 \rightarrow$ zamiana.

Po przejściu 2	3	1	4	5	8
indeks	0	1	2	3	4

PRZEJŚCIE 3 ($i = 2$):

$3 > 1 \rightarrow$ zamiana. $3 < 4 \rightarrow$ bez zamiany.

Po przejściu 3	1	3	4	5	8
indeks	0	1	2	3	4

PRZEJŚCIE 4 ($i = 3$):

$1 < 3 \rightarrow$ bez zamiany. Brak zamian = tablica już posortowana!

Wynik końcowy ✓	1	3	4	5	8
indeks	0	1	2	3	4

2.3. Kod w języku Java

KOD JAVA — Bubble Sort

```
public static void bubbleSort(int[] tab) {  
    int n = tab.length;
```

```

    for (int i = 0; i < n - 1; i++) {           // n-1 przejść
        for (int j = 0; j < n - 1 - i; j++) {   // każde przejście krócej o
1
            if (tab[j] > tab[j + 1]) {           // zła kolejność?
                // zamiana sąsiednich elementów
                int temp = tab[j];
                tab[j] = tab[j + 1];
                tab[j + 1] = temp;
            }
        }
    }
}

// Przykład użycia:
int[] dane = {5, 3, 8, 1, 4};
bubbleSort(dane);
// Wynik: [1, 3, 4, 5, 8]

```

2.4. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność czasowa (śr.)	$O(n^2)$	Dwie zagnieżdżone pętle
Złożoność czasowa (naj.)	$O(n)$	Gdy tablica już posortowana
Złożoność pamięciowa	$O(1)$	Nie potrzebuje dodatkowej tablicy
Stabilność	TAK — stabilny	Równe elementy zachowują kolejność
Zastosowanie	Tablice małe/edukacja	Zbyt wolny dla dużych danych

Zapamiętaj na egzamin!

Bubble Sort: $O(n^2)$ — dwie zagnieżdżone pętle, $n-1$ przejść
 Po każdym przejściu jeden element więcej jest na swoim miejscu (od końca)
 Jest stabilny — równe elementy nie zmieniają kolejności względem siebie
 Jego zaletą jest prostota, wadą — wolność przy dużych danych

3. Sortowanie przez wybieranie (Selection Sort)

3.1. Idea algorytmu

Sortowanie przez wybieranie działa inaczej niż bąbelkowe. Zamiast porównywać sąsiednie elementy i stopniowo przepychać duże wartości na koniec, tutaj za każdym razem szukamy NAJMNIEJSZEGO elementu w nieposortowanej części i wstawiamy go na właściwe miejsce.

Analogia z życia

Wyobraź sobie 5 kart z liczbami rozłożonych na stole.
Patrzysz na wszystkie karty i wybierasz tę z NAJMNIEJSZĄ liczbą.
Wkładasz ją na pierwsze miejsce w ręce.
Patrzysz na POZOSTAŁE karty i znowu wybierasz najmniejszą.
Wkładasz ją na drugie miejsce.
Powtarzasz, aż masz wszystkie karty posortowane w ręce.

3.2. Przykład krok po kroku

Mamy tablicę: [64, 25, 12, 22, 11]

PRZEJŚCIE 1: Szukamy minimum w całej tablicy.

Tablica początkowa	64	25	12	22	11
indeks	0	1	2	3	4

Minimum = 11 (na indeksie 4). Zamieniamy z indeksem 0.

Po zamianie [0] z [4]	11	25	12	22	64
indeks	0	1	2	3	4

PRZEJŚCIE 2: Szukamy minimum w tablicy od indeksu 1.

Minimum = 12 (na indeksie 2). Zamieniamy z indeksem 1.

Po zamianie [1] z [2]	11	12	25	22	64
indeks	0	1	2	3	4

PRZEJŚCIE 3: Szukamy minimum od indeksu 2.

Minimum = 22 (na indeksie 3). Zamieniamy z indeksem 2.

Po zamianie [2] z [3]	11	12	22	25	64
-----------------------	----	----	----	----	----

indeks	0	1	2	3	4
--------	---	---	---	---	---

PRZEJŚCIE 4: Szukamy minimum od indeksu 3.

Minimum = 25 (już na indeksie 3). Brak zamiany.

Wynik końcowy ✓	11	12	22	25	64
indeks	0	1	2	3	4

3.3. Kod w języku Java

KOD JAVA – Selection Sort

```
public static void selectionSort(int[] tab) {
    int n = tab.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i; // zakładamy minimum na
        pozycji i
        for (int j = i + 1; j < n; j++) { // szukamy minimum w
        reszcie
            if (tab[j] < tab[minIdx]) {
                minIdx = j; // zapamiętaj indeks
            }
        }
        if (minIdx != i) { // wykonaj zamianę tylko
        gdy trzeba
            int temp = tab[i];
            tab[i] = tab[minIdx];
            tab[minIdx] = temp;
        }
    }
}

// Przykład użycia:
int[] dane = {64, 25, 12, 22, 11};
selectionSort(dane);
// Wynik: [11, 12, 22, 25, 64]
```

3.4. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność czasowa (każdy)	$O(n^2)$	Zawsze dwie pętle, niezależnie od danych
Złożoność pamięciowa	$O(1)$	Tylko jedna zmienna pomocnicza (min_idx)

Stabilność	NIE — niestabilny	Zamiana może zmienić kolejność równych elem.
Liczba zamian	$O(n)$ — minimalna	Nigdy więcej niż $n-1$ zamian!
Zastosowanie	Gdy koszt zamiany jest duży	Minimalizuje liczbę przestawień

Zapamiętaj na egzamin!

Selection Sort: $O(n^2)$ zawsze — nie ma lepszego przypadku

Minimum zamian: co najwyżej $n-1$ zamian (to jego zaleta!)

Jest NIEstabilny — może zmienić kolejność równych elementów

Różnica od Bubble Sort: Selection szuka minimum i wstawia raz, Bubble zamienia wiele razy

4. Sortowanie przez wstawianie (Insertion Sort)]

4.1. Idea algorytmu

Sortowanie przez wstawianie jest zainspirowane tym, jak gracze w karty układają karty w ręce. Bierzymy elementy jeden po jednym i wstawiamy każdy na właściwe miejsce w już posortowanej części.

Analogia z życia

Wyobraź sobie, że grasz w karty i dobierasz kartę z talii po jednej.

Kiedy bierzesz nową kartę, wkładasz ją w odpowiednie miejsce w ręce.

Lewa część ręki jest zawsze posortowana.

Prawa część to karty jeszcze nieprzejrzane.

Nową kartę wsuwa się od prawej strony posortowanej części, przesuwając większe karty w prawo.

4.2. Przykład krok po kroku

Mamy tablicę: [5, 2, 4, 6, 1]. Zaznaczone komórki to element aktualnie wstawiany.

Krok 1: Zaczynamy od drugiego elementu (indeks 1).

Bierzemy: 2	5	2	4	6	1
indeks	0	1	2	3	4

$2 < 5$, więc przesuwamy 5 w prawo i wstawiamy 2 na indeks 0.

Po wstawieniu	2	5	4	6	1
indeks	0	1	2	3	4

Krok 2: Bierzymy element na indeksie 2.

Bierzemy: 4	2	5	4	6	1
indeks	0	1	2	3	4

$4 < 5$, więc przesuwamy 5 w prawo. $4 > 2$, więc wstawiamy 4 na indeks 1.

Po wstawieniu	2	4	5	6	1
indeks	0	1	2	3	4

Krok 3: Bierzymy element na indeksie 3.

Bierzemy: 6	2	4	5	6	1
indeks	0	1	2	3	4

6 > 5, więc nie przesuwamy nic. 6 zostaje na miejscu.

Krok 4: Bierzemy element na indeksie 4.

Bierzemy: 1	2	4	5	6	1
indeks	0	1	2	3	4

1 < 6, 1 < 5, 1 < 4, 1 < 2 — wszystko przesuwamy w prawo, 1 trafia na indeks 0.

Wynik końcowy ✓	1	2	4	5	6
indeks	0	1	2	3	4

4.3. Kod w języku Java

KOD JAVA — Insertion Sort

```
public static void insertionSort(int[] tab) {
    int n = tab.length;
    for (int i = 1; i < n; i++) {           // zaczynamy od drugiego
    elementu
        int klucz = tab[i];                 // zapamiętaj bieżący element
        int j = i - 1;                      // patrz na lewo
        // przesun większe elementy o jedno miejsce w prawo
        while (j >= 0 && tab[j] > klucz) {
            tab[j + 1] = tab[j];
            j--;
        }
        tab[j + 1] = klucz;                 // wstaw element na właściwe
    miejsce
    }
}

// Przykład użycia:
int[] dane = {5, 2, 4, 6, 1};
insertionSort(dane);
// Wynik: [1, 2, 4, 5, 6]
```

4.4. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność czasowa (śr.)	$O(n^2)$	Dwie pętle: zewnętrzna i while
Złożoność czasowa (najlep.)	$O(n)$ — najlepszy!	Gdy tablica już posortowana — tylko 1 przejście
Złożoność pamięciowa	$O(1)$	Tylko zmienna 'klucz'
Stabilność	TAK — stabilny	Zachowuje kolejność równych elementów

Zastosowanie	Małe lub prawie posortowane dane	Często lepszy niż Bubble i Selection
--------------	----------------------------------	--------------------------------------

Zapamiętaj na egzamin!

Insertion Sort jest optymalny dla danych prawie posortowanych — $O(n)$

Jest stabilny, ma złożoność $O(n^2)$ średnio, ale $O(n)$ w najlepszym przypadku

Klucz (key) = element aktualnie wstawiany — ważne pojęcie w opisach tego algorytmu

Lewa część tablicy jest zawsze posortowana — to cecha charakterystyczna

5. Sortowanie przez scalanie (Merge Sort)

5.1. Strategia: Dziel i zwyciężaj

Merge Sort używa zupełnie innej strategii niż poprzednie algorytmy. Zamiast porównywać sąsiednie elementy, stosuje podejście 'dziel i zwyciężaj':

Trzy kroki strategii 'Dziel i zwyciężaj'

KROK 1 — PODZIEL: Podziel tablicę na dwie równe połowy.

KROK 2 — ZWYCIĘŻAJ: Sortuj każdą połowę (rekurencyjnie tą samą metodą).

KROK 3 — SCALA: Połącz dwie posortowane połowy w jedną posortowaną tablicę.

Analogia z życia

Masz 8 kartek z liczbami do posortowania.

Dzielisz je na dwie kupki po 4.

Każdą kupkę dzielisz dalej na dwie po 2, potem na pojedyncze kartki.

Jedna kartka jest już 'posortowana' (trivialnie).

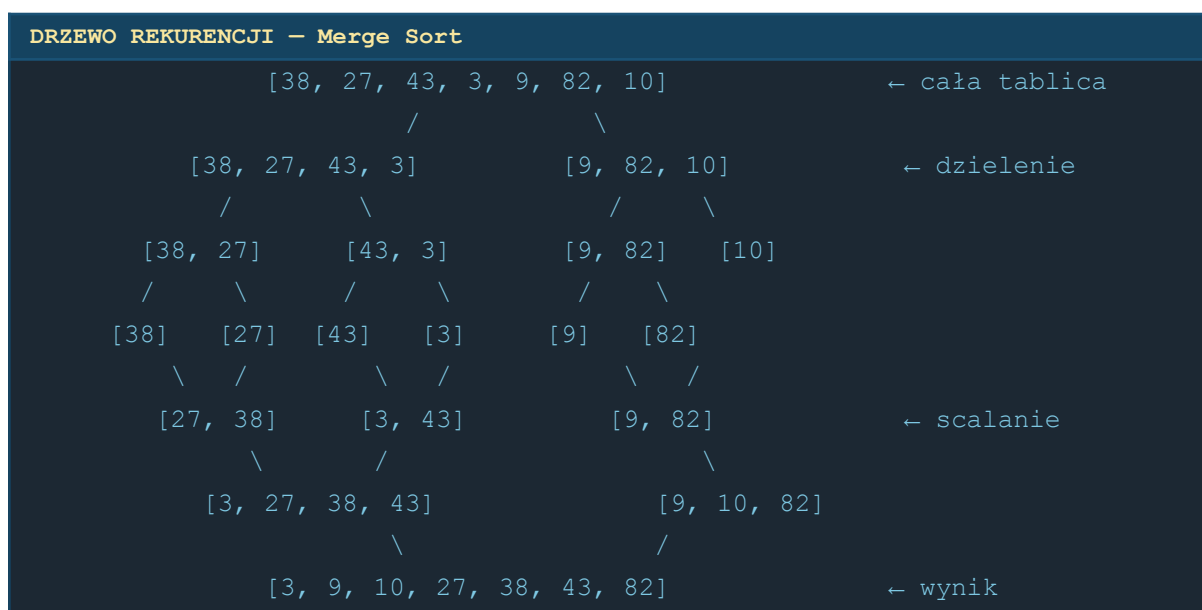
Teraz scalasz pary: bierzesz dwie pojedyncze kartki i układasz w kolejności.

Scalasz czwórki z par, potem ósemkę z czwórek.

Na końcu masz jedną posortowaną kupkę.

5.2. Przykład — drzewo rekurencji

Tablice: [38, 27, 43, 3, 9, 82, 10]



5.3. Jak działa scalanie (merge)?

To najważniejsza część algorytmu. Mamy dwie POSORTOWANE tablice i musimy je złączyć w jedną posortowaną.

Przykład: scalamy [3, 27, 38, 43] i [9, 10, 82]

Lewa	Prawa	Porównanie	Wynik (dodajemy do nowej tablicy)
3	9	$3 < 9$	Dodajemy 3. Lewa przesuwa się.
27	9	$9 < 27$	Dodajemy 9. Prawa przesuwa się.
27	10	$10 < 27$	Dodajemy 10. Prawa przesuwa się.
27	82	$27 < 82$	Dodajemy 27. Lewa przesuwa się.
38	82	$38 < 82$	Dodajemy 38. Lewa przesuwa się.
43	82	$43 < 82$	Dodajemy 43. Lewa przesuwa się.
(koniec)	82	Lewa skończona	Dodajemy resztę prawej: 82.

Wynik: [3, 9, 10, 27, 38, 43, 82] ✓

5.4. Kod w języku Java

KOD JAVA — Merge Sort

```
public static int[] mergeSort(int[] tab) {
    if (tab.length <= 1) return tab; // tablica 0 lub 1-elementowa jest posortowana

    // DZIEL: znajdź środek i podziel tablicę
    int srodek = tab.length / 2;
    int[] lewa = Arrays.copyOfRange(tab, 0, srodek);
    int[] prawa = Arrays.copyOfRange(tab, srodek, tab.length);

    lewa = mergeSort(lewa); // rekurencyjnie sortuj lewą część
    prawa = mergeSort(prawa); // rekurencyjnie sortuj prawą część

    return scal(lewa, prawa); // SCAL: połącz obie posortowane połowy
}

public static int[] scal(int[] lewa, int[] prawa) {
    int[] wynik = new int[lewa.length + prawa.length];
    int i = 0, j = 0, k = 0;
    while (i < lewa.length && j < prawa.length) {
        if (lewa[i] <= prawa[j]) { // mniejszy element idzie do wyniku
            wynik[k++] = lewa[i++];
        } else {
            wynik[k++] = prawa[j++];
        }
    }
}
```

```

    }
    while (i < lewa.length) wynik[k++] = lewa[i++]; // reszta lewej
    while (j < prawa.length) wynik[k++] = prawa[j++]; // reszta prawej
    return wynik;
}

// Przykład użycia (wymaga import java.util.Arrays):
int[] dane = {38, 27, 43, 3, 9, 82, 10};
int[] wynik = mergeSort(dane);
// Wynik: [3, 9, 10, 27, 38, 43, 82]

```

5.5. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność czasowa	$O(n \log n)$ — zawsze	$\log n$ poziomów $\times$ n pracy na każdym
Złożoność pamięciowa	$O(n)$	Wymaga tymczasowych tablic podczas scalania
Stabilność	TAK — stabilny	$\text{lewa}[i] \leq \text{prawa}[j]$ zachowuje równe elementy
Zastosowanie	Duże dane, sortowanie stabilne	Używany w Javie (Arrays.sort) i innych językach

Zapamiętaj na egzamin!

Merge Sort: $O(n \log n)$ zawsze — niezależnie od danych wejściowych
 Wymaga $O(n)$ dodatkowej pamięci — to jedyna wada
 Jest stabilny i bardzo szybki dla dużych zbiorów danych
 Strategia: dziel i zwyciężaj — rekurencja + scalanie

6. Sortowanie szybkie (Quick Sort)

6.1. Idea algorytmu

Quick Sort (sortowanie szybkie) to w praktyce jeden z najszybszych algorytmów sortowania. Też używa strategii 'dziel i zwyciężaj', ale inaczej niż Merge Sort.

Kluczowe pojęcie: Pivot (element osiowy)

Pivot to element tablicy wybrany jako 'punkt odniesienia'.

Wszystkie elementy MNIEJSZE od pivotu trafiają PO JEGO LEWEJ stronie.

Wszystkie elementy WIĘKSZE od pivotu trafiają PO JEGO PRAWEJ stronie.

Po tej operacji pivot jest już na swojej DOCELOWEJ pozycji w posortowanej tablicy!

Następnie rekurencyjnie sortujemy lewą i prawą część.

Analogia z życia

Wyobraź sobie, że uczniowie ustawiają się według wzrostu.

Nauczyciel wskazuje jednego ucznia jako 'wzorzec' (pivot).

Wszyscy niżsi od wzorca ustawiają się po jego lewej stronie.

Wszyscy wyżsi — po prawej stronie.

Wzorzec jest teraz na swoim miejscu!

Teraz powtarzamy to samo dla grupy po lewej i grupy po prawej.

6.2. Przykład krok po kroku

Mamy tablicę: [10, 7, 8, 9, 1, 5]. Wybieramy ostatni element jako pivot.

KROK 1: Pivot = 5 (ostatni element).

Tablica, pivot=5	10	7	8	9	1	5
indeks	0	1	2	3	4	5

Partycjonujemy: elementy mniejsze niż 5 trafiają na lewo, większe na prawo.

Wynik po partycjonowaniu (pivot 5 jest na indeksie 1):

Po partycjon owaniu	1	5	10	7	8	9
indeks	0	1	2	3	4	5

Pivot 5 jest na swoim docelowym miejscu! Teraz rekurencyjnie sortujemy [1] i [10, 7, 8, 9].

KROK 2: Sortujemy [10, 7, 8, 9]. Pivot = 9.

Podtablica	10	7	8	9
indeks	0	1	2	3

Po partycjonowaniu: [7, 8, 9, 10]. Pivot 9 na indeksie 2.

Po partycjonowaniu	7	8	9	10
indeks	0	1	2	3

KROK 3: Sortujemy [7, 8]. Pivot = 8.

$7 < 8 \rightarrow 7$ po lewej. [7, 8] — już posortowane.

Wynik końcowy: [1, 5, 7, 8, 9, 10] ✓

Wynik końcowy ✓	1	5	7	8	9	10
indeks	0	1	2	3	4	5

6.3. Kod w języku Java

KOD JAVA — Quick Sort

```
public static void quickSort(int[] tab, int lewy, int prawy) {
    if (lewy < prawy) {
        int pivotIdx = partycjonuj(tab, lewy, prawy);
        quickSort(tab, lewy, pivotIdx - 1); // sortuj lewą część
        quickSort(tab, pivotIdx + 1, prawy); // sortuj prawą część
    }
}

public static int partycjonuj(int[] tab, int lewy, int prawy) {
    int pivot = tab[prawy]; // ostatni element jako pivot
    int i = lewy - 1;
    for (int j = lewy; j < prawy; j++) {
        if (tab[j] <= pivot) { // element mniejszy/równy pivot — idzie na lewo
            i++;
            int temp = tab[i]; tab[i] = tab[j]; tab[j] = temp;
        }
    }
    // postaw pivot na właściwym miejscu
    int temp = tab[i + 1]; tab[i + 1] = tab[prawy]; tab[prawy] = temp;
    return i + 1; // zwróć indeks pivotu
}

// Przykład użycia:
```



```
int[] dane = {10, 7, 8, 9, 1, 5};
quickSort(dane, 0, dane.length - 1);
// Wynik: [1, 5, 7, 8, 9, 10]
```

6.4. Złożoność i cechy

Cecha	Wartość	Wyjaśnienie
Złożoność (śr./najlep.)	$O(n \log n)$ — szybko!	Pivot dzieli tablicę mniej więcej na pół
Złożoność (najgorszy)	$O(n^2)$ — wolno!	Gdy tablica posortowana i pivot=ostatni
Złożoność pamięciowa	$O(\log n)$	Stos rekurencji (mniej niż Merge Sort)
Stabilność	NIE — niestabilny	Partycjonowanie może zmienić kolejność równych
Zastosowanie	Praktyczne sortowanie, duże dane	Najczęściej używany w praktyce

⚠ Zapamiętaj na egzamin!

Quick Sort: $O(n \log n)$ średnio, ale $O(n^2)$ dla posortowanych danych (zły pivot)

Jest NIEstabilny — pivot może zmienić kolejność równych elementów

Najgorszy przypadek: tablica już posortowana + pivot jako ostatni/pierwszy element

W praktyce szybszy niż Merge Sort mimo tej samej złożoności (mniej operacji na pamięci)

7. Porównanie wszystkich algorytmów

Na egzaminie INF.04 możesz zostać zapytany o porównanie algorytmów. Oto kompletna tabela:

Algorytm	Czas (śr.)	Czas (naj.)	Pamięć	Stabilny
Bubble Sort	$O(n^2)$	$O(n)$	$O(1)$	TAK
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$	NIE
Insertion Sort	$O(n^2)$	$O(n)$	$O(1)$	TAK
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	TAK
Quick Sort	$O(n \log n)$	$O(n^2)!$	$O(\log n)$	NIE

7.1. Kiedy używać którego algorytmu?

Sytuacja	Najlepszy wybór	Dlaczego
Mała tablica (< 20 elementów)	Insertion Sort	Prosty, mało narzutu, dobry dla małych n
Prawie posortowana tablica	Insertion Sort	W najlepszym przypadku $O(n)$
Duże dane, potrzeba stabilności	Merge Sort	$O(n \log n)$ zawsze, stabilny
Duże dane, zależy nam na szybkości	Quick Sort	Najszybszy w praktyce (dobre stałe)
Minimalna liczba zamian elementów	Selection Sort	Co najwyżej $n-1$ zamian
Edukacja / egzamin — prostota	Bubble Sort	Najprostszy do wyjaśnienia

7.2. Kluczowe różnice — jak je zapamiętać?

Mnemotechnika — jak zapamiętać każdy algorytm

BUBBLE — 'Bąble' dużych wartości wypływają na koniec. Sąsiednie pary.

SELECTION — 'Wybieranie' minimum i wstawianie na właściwe miejsce. Min zamian.

INSERTION — 'Wstawianie' jak kart w ręce. Lewy = posortowany, prawy = do zrobienia.

MERGE — 'Scalanie'. Dziel na pół aż do 1 elementu, potem scalaj. Zawsze $n \log n$.

QUICK — 'Szybki' dzięki pivotowi. Lewo < pivot < prawo. Może być $O(n^2)$.